

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2016 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 1 (20 iulie 2016)	2
SUBIECTUL I (30 de puncte)	2
1. Algoritmul de parcurgere în lățime (BFS)	2
2. Rețele: Serviciul E-mail	2
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Cuvinte cu același șir vocalic	3
2. Algoritm eficient: Reuniune de intervale (Merge Intervals - $O(N \log N)$ timp)	5

I. Rezolvare Titularizare Varianta 1 (20 iulie 2016)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Algoritmul de parcurgere în lăţime (BFS)

• Definiţii:

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este mulţimea nodurilor şi E este o mulţime de perechi neordonate de noduri (muchii).
- **Graf orientat:** Pereche ordonată $G = (V, A)$, unde A este o mulţime de perechi ordonate de noduri (arce).
- **Adiacenţă:** Două noduri sunt adiacente dacă există o muchie (sau un arc) care le conectează direct.

- **Descriere BFS:** Foloseşte o coadă (Queue) pentru a stoca nodurile vizitate a căror vecinătate nu a fost încă investigată. Se pleacă de la un nod sursă, se adaugă în coadă şi se marchează ca vizitat. Cât timp coada nu este vidă, se extrage primul nod, i se parcurg toţi vecinii nevizitaţi, se adaugă la rândul lor în coadă şi se marchează ca vizitaţi.

- **Trace:** Execuţia pe un graf cu 10 noduri.

• Problemă (BFS):

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
void bfs(int start, const vector<vector<int>> &adj, int n) {
    vector<bool> viz(n + 1, false);
    queue<int> Q;
    viz[start] = true;
    Q.push(start);
    while (!Q.empty()) {
        int u = Q.front(); Q.pop();
        cout << u << " ";
        for (int v : adj[u]) {
            if (!viz[v]) {
                viz[v] = true;
                Q.push(v);
            }
        }
    }
}
```

2. Reţele: Serviciul E-mail

- **Funcţii reţea:** Partajare fişiere, partajare echipamente hardware, servicii de comunicare/colaborare.
- **Protocoale e-mail:** SMTP (Simple Mail Transfer Protocol - trimitere), POP3/IMAP (recepţiune).
- **Aplicaţii client:** Microsoft Outlook, Mozilla Thunderbird.
- **Structură adresă:** utilizator@domeniu.extensie.
- **Structură mesaj:** Antet (From, To, Subject), Corp mesaj (Body), Ataşamente.
- **Facilităţi:** Trimitere asincronă, trimitere multiplă (Cc/Bcc).

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Cuvinte cu același șir vocalic

Soluție C++:

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
#include <map>
using namespace std;

void vocalic(const string &s, string &sv) {
    sv = "";
    for (char c : s) {
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
            sv += c;
        }
    }
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string cuvant;
    map<string, int> frecventa;

    while (ss >> cuvant) {
        string sv;
        vocalic(cuvant, sv);
        frecventa[sv]++;
    }

    bool gasit = false;
    for (auto &pair : frecventa) {
        if (pair.second >= 2) {
            gasit = true;
            break;
        }
    }

    if (gasit) cout << "DA\n";
    else cout << "NU\n";
    return 0;
}
```

Soluție Pascal:

```
program CuvinteVocalice;
var
    text_input, cuvant: string;
    cuvinte, vocale: array[1..100] of string;
    nr_cuvinte, i, j: integer;
```

```

gasit: boolean;

procedure vocalic(s: string; var sv: string);
var idx: integer;
begin
    sv := '';
    for idx := 1 to length(s) do
    begin
        if (s[idx] = 'a') or (s[idx] = 'e') or (s[idx] = 'i') or (s[idx] = 'o') or
(s[idx] = 'u') then
            sv := sv + s[idx];
        end;
    end;

begin
    readln(text_input);
    text_input := text_input + ' ';
    nr_cuvinte := 0;
    cuvant := '';
    for i := 1 to length(text_input) do
    begin
        if text_input[i] <> ' ' then
            cuvant := cuvant + text_input[i]
        else
            begin
                if length(cuvant) > 0 then
                begin
                    nr_cuvinte := nr_cuvinte + 1;
                    cuvinte[nr_cuvinte] := cuvant;
                    vocalic(cuvant, vocale[nr_cuvinte]);
                    cuvant := '';
                end;
            end;
        end;
    end;

    gasit := false;
    for i := 1 to nr_cuvinte do
    begin
        for j := i + 1 to nr_cuvinte do
        begin
            if (vocale[i] = vocale[j]) then
            begin
                gasit := true;
                break;
            end;
        end;
        if gasit then break;
    end;

    if gasit then writeln('DA')
    else writeln('NU');
end.

```

2. Algoritm eficient: Reuniune de intervale (Merge Intervals - $O(N \log N)$ timp)

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

struct Interval {
    int st, dr;
};

bool cmp(const Interval &a, const Interval &b) {
    if (a.st != b.st) return a.st < b.st;
    return a.dr < b.dr;
}

int main() {
    ifstream fin("titu.in");
    int n;
    if (!(fin >> n)) return 0;
    vector<Interval> intervals(n);
    for (int i = 0; i < n; ++i) {
        fin >> intervals[i].st >> intervals[i].dr;
    }
    fin.close();

    sort(intervals.begin(), intervals.end(), cmp);

    int count = 0;
    if (n > 0) {
        int current_st = intervals[0].st;
        int current_dr = intervals[0].dr;
        for (int i = 1; i < n; ++i) {
            if (intervals[i].st <= current_dr) {
                current_dr = max(current_dr, intervals[i].dr);
            } else {
                count++;
                current_st = intervals[i].st;
                current_dr = intervals[i].dr;
            }
        }
        count++;
    }
    cout << count << endl;
    return 0;
}
```

Soluție Pascal:

```
program ReuniuneIntervale;
type
    Interval = record
        st, dr: integer;
    end;
var
```

```

fin: text;
intervals: array[1..10000] of Interval;
n, i, count: integer;
curr_st, curr_dr: integer;

procedure QuickSort(l, r: integer);
var
    i_idx, j_idx: integer;
    pivot, tmp: Interval;
begin
    i_idx := l; j_idx := r;
    pivot := intervals[(l + r) div 2];
    repeat
        while (intervals[i_idx].st < pivot.st) or ((intervals[i_idx].st = pivot.st) and
(intervals[i_idx].dr < pivot.dr)) do i_idx := i_idx + 1;
        while (intervals[j_idx].st > pivot.st) or ((intervals[j_idx].st = pivot.st) and
(intervals[j_idx].dr > pivot.dr)) do j_idx := j_idx - 1;
        if i_idx <= j_idx then
            begin
                tmp := intervals[i_idx]; intervals[i_idx] := intervals[j_idx];
intervals[j_idx] := tmp;
                i_idx := i_idx + 1; j_idx := j_idx - 1;
            end;
        until i_idx > j_idx;
        if l < j_idx then QuickSort(l, j_idx);
        if i_idx < r then QuickSort(i_idx, r);
    end;

begin
    assign(fin, 'titu.in'); reset(fin);
    read(fin, n);
    for i := 1 to n do read(fin, intervals[i].st, intervals[i].dr);
    close(fin);

    QuickSort(1, n);

    count := 0;
    if n > 0 then
        begin
            curr_st := intervals[1].st;
            curr_dr := intervals[1].dr;
            for i := 2 to n do
                begin
                    if intervals[i].st <= curr_dr then
                        begin
                            if intervals[i].dr > curr_dr then curr_dr := intervals[i].dr;
                        end
                    else
                        begin
                            count := count + 1;
                            curr_st := intervals[i].st;
                            curr_dr := intervals[i].dr;
                        end;
                    end;
                count := count + 1;
            end;
        end;
    end;

```

```
writeln(count);  
end.
```